

Профилирование старта контроллера android

В связи с тем, что периодически возникают вопросы и трудности при профилировании старта android приложения в разрезе контроллера, ниже представлен проверенный метод, который позволяет эффективно выполнить профилирование, однако требует использования ADB (Android Debug Bridge).

1. Соберите ваше приложение со включенным флагом SBIS_OPTIMIZATIONS.

Принудительное включение оптимизации для контроллера нужно делать через изменение строчки здесь (если не планируется собирать ui в release конфигурации):

в application module/android-configs

в контроллере android-configs

https://git.sbis.ru/mobileworkspace/configs/android-configs/-/blob/5cbdab41154641eb507718f4c4fba1e75ddbd1ba/gradle_configs/gradle_settings/settings-plugin/src/main/groovy/ru/tensor/gradle/convention/controller/ControllerCfgPlugin.groovy#L372

```
debug {
    externalNativeBuild {
        cmake {
            arguments += buildCmakeOptimizationOption(true)
        }
    }
}
```

2. Установите приложение на девайс или эмулятор с помощью андроид студии или вручную.

3. Перейдите в каталог в котором расположен adb и выполните следующую команду:

```
./adb push
```

```
<корневая_папка_приложения>/app/build/intermediates/merged_native_libs/debug/mergeDebugNativeLibs/out/lib/<ваша_архитектура>/*.so /data/local/tmp/native_libs
```



Пример итоговой команды:

```
./adb push /home/local/TENSOR-CORP/dr.spirov/projects/repos/mobile/retail/application/app/build/intermediates/merged_native_libs/debug/mergeDebugNativeLibs/out/lib/arm64-v8a/*.so /data/local/tmp/native_libs
```

Если на этом шаге adb выдает, что никакой девайс не подключен, подключитесь через "./adb connect".

Если при копировании возникает ошибка, что целевая директория не существует, то её нужно создать через андроид студию или с помощью `adb shell mkdir`

```
adb shell mkdir /data/local/tmp/native_libs
```

4. Подготовьтесь к выполнению профилирования, если требуется замерять какой-то конкретный сценарий.

Для старта профилирования выполните команду:

```
./adb shell simpleperf record --app <имя_пакета_приложения> --symfs /data/local/tmp/native_libs -g --user-buffer-size 3072M --duration <продолжительность_записи_профиля> -o /data/local/tmp/perf1.data
```



Пример команды для retail:

```
./adb shell simpleperf record --app ru.tensor.saby.retail.debug --symfs /data/local/tmp/native_libs -g --user-buffer-size 3072M --duration 15 -o /data/local/tmp/perf1.data
```

- 4.1. На этом шаге могут возникнуть непредвиденные ошибки, особенно если профилирование выполняется на реальном устройстве:

- Unknown argument "user-buffer-size" и подобное - на устройстве старая версия simpleperf, она не поддерживает явное резервирование кеша для профиля и может в целом работать неудовлетворительно. Рекомендуется попробовать скопировать simpleperf из ndk и использовать его. simpleperf находится здесь "`<ваш_путь_до_ndk>/28.2.13676358/simpleperf/bin/android/arm64/simpleperf`".



Команда для копирования:

```
./adb push /home/dr.spirov/Android/Sdk/ndk/28.2.13676358/simpleperf/bin/android/arm64/simpleperf
/data/local/tmp/simpleperf
```

Команда для запуска simpleperf:

```
./adb shell /data/local/tmp/simpleperf record <остальные_аргументы>
```

В некоторых случаях это не исправит ошибку "unknown argument user-buffer-size", т.к. для записи сэмплов используется встроенная утилита "simpleperf_app_runner", которую уже подменить нельзя. В таком случае может помочь использование аргументов "--no-cut-samples", "-m 1024", "--include-thread-name DefaultDispatch" (имя потока, для которого собирается профиль, если известно заранее). Также может быть полезно изменение частоты записи сэмплов (параметр -f).

- Нет никаких ошибок, но ничего не записывает; сообщение вида record failed без объяснения причины.

Выполните `./adb shell` и вызывайте команду оттуда, теперь ошибка должна отображаться в выводе консоли. Также может иметь смысл передать "--log verbose" в аргументы simpleperf.

- "app is not debuggable" или "app is not profilable". Вероятно приложение собрано в release конфигурации и там не выставлена настройка "debuggable true". Указать её можно в build.gradle приложения там же где указывался "signingConfig signingConfigs.debug" для локальной сборки релиза. После этого потребуется собрать и переустановить приложение.

5. Выполните действия, которые вы будете профилировать. Профилирование автоматически завершится через столько секунд, сколько указано в --duration.

6. Выполните команду, чтобы скачать результат профилирования:

```
./adb pull /data/local/tmp/perf1.data <путь_куда_сохранить_профиль>
```



```
./adb pull /data/local/tmp/perf1.data /home/local/TENSOR-
CORP/dr.spirov/Android/Sdk/ndk/27.2.12479018/simpleperf
```

7. Перейдите в каталог с simpleperf:

simpleperf поставляется с ndk

```
<путь до ndk>/ndk/27.2.12479018/simpleperf
```

Выполните команду:

```
python3.11 report_html.py -i <путь_до_данных> -o <куда_сохранить_результат>
```



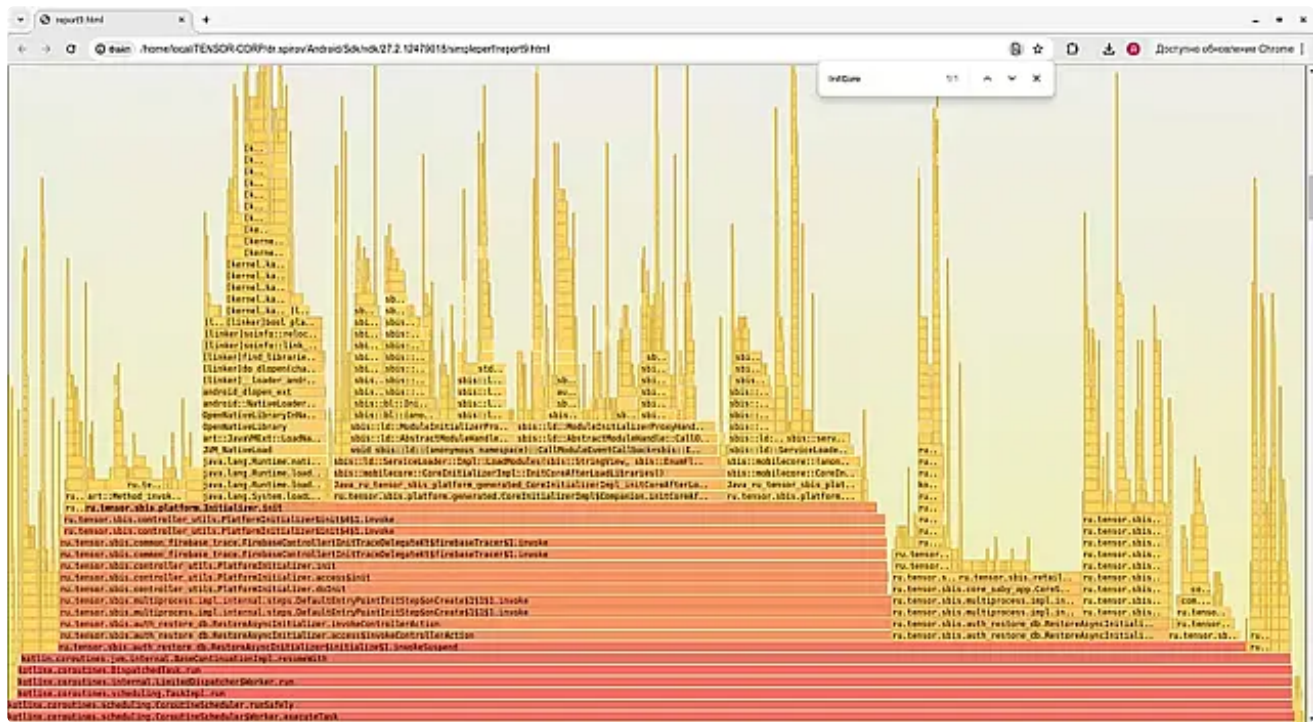
Если данные сразу копировались в папку с simpleperf, то команда будет:

```
python3.11 report_html.py -i perf1.data -o report1.html
```



При работе на Windows report_html.py может выдавать ошибку, тогда нужно проверить что используется 64-бит питон.

8. После этого будет построен отчет в виде html файла, содержащий профиль.



Если вместо фреймов контроллера отображаются адреса в формате "sbis-app-controller.so + xxxxxx", это может указывать на загрузку некорректных файлов символов, которые профилировщик не смог распознать. Также проблема может быть в самом устройстве или в его версии `simpleperf`, если профилирование на физическом девайсе. В данном случае отсутствуют конкретные рекомендации по устранению проблемы, поэтому рекомендуется использовать другое устройство.

<https://online.sbis.ru/news/c33deade-d9dc-4616-9882-4fd75058ac8a>